Infrastructure As Code (IAC)

Building a 12 factor app

Brice Ekane
brice.ekane@univ-rennes.fr
https://github.com/bekane/tlc.git
2025-2026

Cours repris de Pr. Olivier Barais

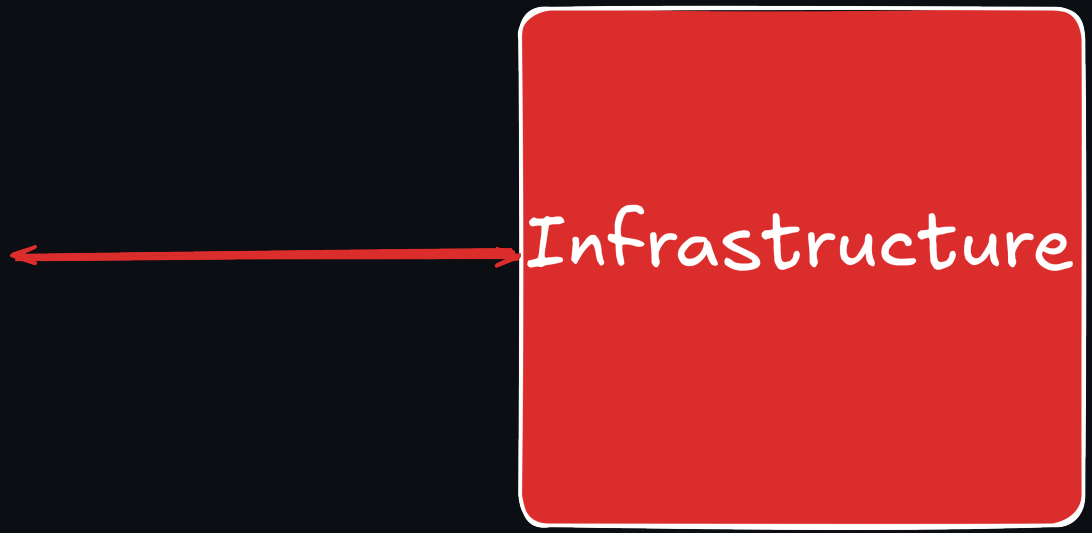
12 Factors in a nutshell

– A methodology
– Best Practices
– Manifesto

https://12factor.net/ by Heroku

Why 12 factor?

Define the contract between applications and infrastructure



Application InfrastructureWhat is a Twelve-Factor App?

In the modern era, software is commonly delivered as a service: called web apps, or software-as-a- service. The twelve-factor app is a methodology for building software-as-a-service apps that: Use **declarative** formats for setup automation, to minimize time and cost for new developers joining the project; Have a **clean contract** with the underlying operating system, offering **maximum portability** between execution environments; Are suitable for **deployment** on modern **cloud platforms**, obviating the need for servers and systems administration; **Minimize divergence** between development and production, enabling **continuous deployment** for maximum agility; And can **scale up** without significant changes to tooling, architecture, or development practices. The twelve-factor methodology can be applied to apps written in any programming language, and which use any combination of backing services (database, queue, memory cache, etc).

From https://12factor.net

THE FACTORS

From https://medium.com/cloud-native-architecture-and-design-guide/the-12-factor-app-methodology-a-blueprint-for-modern-software-033ddd7b2dcd

I. Codebase

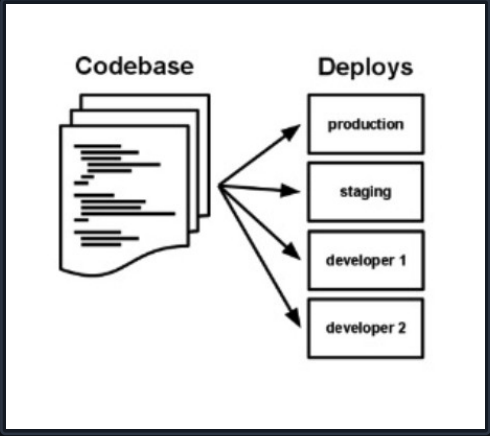
“One codebase tracked in revision control, many deploys.”

- Dedicate smaller teams to individual applications or microservices.
- Following the discipline of single repository for an application forces

the teams to analyze the seams of their application, and identify potential monoliths that should be split off into microservices.

- Use a single source code repository for

a single application (1:1 relation). Deployment stages are different tags/branches

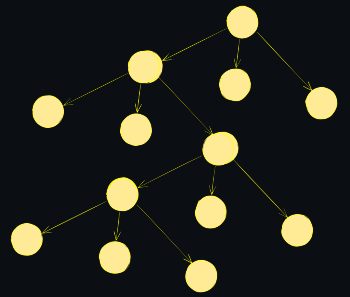


- i.e. use a central git repo (external Github/ GitHub Enterprise also suitable)

II. Dependencies

“Explicitly declare and isolate dependencies”
A cloud-native application does not rely on the pre-existence of dependencies in a deployment target.
Developer Tools declare and isolate dependencies

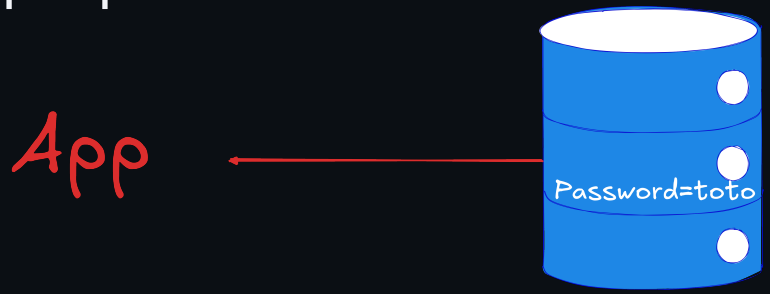
- Maven and Gradle for Java
- Each microservice has its own dependencies declared (e.g. pom.xml)



III. Config

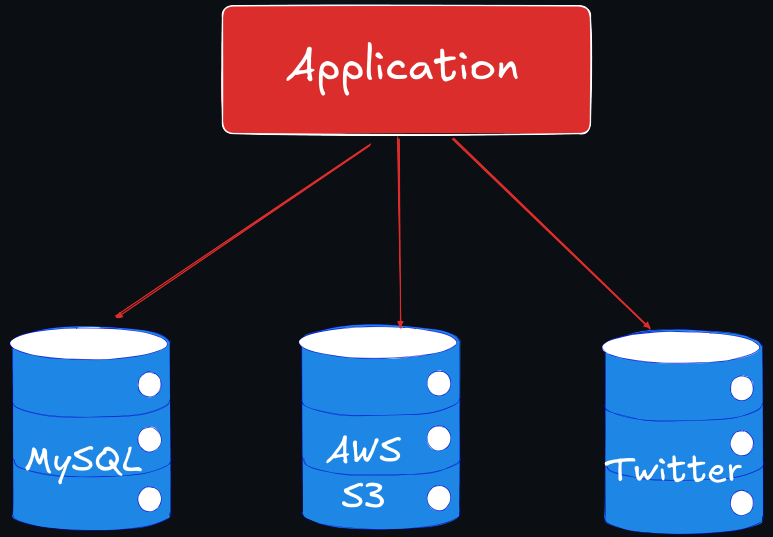
“Store config in the environment”

- Changing config should not need to repackage your application
- Use Kubernetes configmaps and secrets (rather than environment variables) for container services
- Use Vault to inject the config properties into the microservices



IV. Backing services

“Treat backing services as attached resources”



V. Build, release, run

“Strictly separate build and run stages”

- Source code is used in the build stage. Configuration data is added to define a release stage that can be deployed. Any changes in code or config will result in a new build/release
- Needs to be considered in CI pipeline

IBM

- UrbanCode Deploy
- IBM Cloud Continuous Delivery Service

AWS

- AWS CodeBuild
- AWS CodeDeploy
- AWS CodePipeline(not yet integrated with EKS)

Azure

- Visual Studio Team Services (VSTS) (includes git)
- Web App for Containers feature of Azure App Service

VI. Processes

“Execute the app as one or more stateless processes”

Stateless and share-nothing Rest API

VII. Port binding

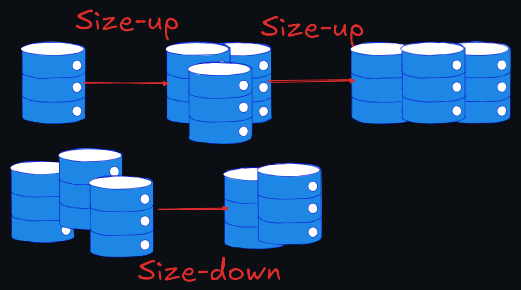
“Export services via port binding”

- Applications are fully self-contained and expose services only through ports. Port assignment is done by the execution environment
- Ingress/service definition of k8s manages mapping of ports
- Use MP Config to inject ports to microservices for chain-up invocations

VIII. Concurrency

“Scale out via the process model”

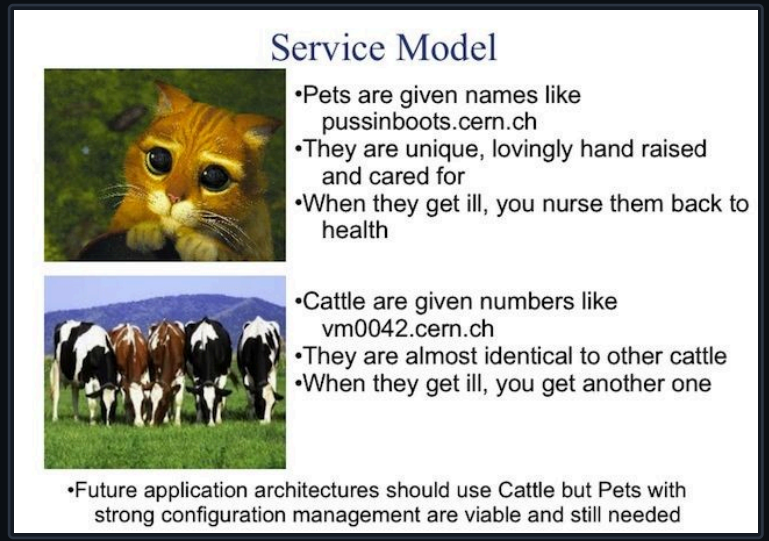
- Applications use processes independent from each other to scale out (allowing for load balancing)
- To be considered in application design
- Cloud autoscaling services: [auto]scaling built into k8s
- Build micorservices



IX. Disposability

“Maximize robustness with fast startup and graceful shutdown”

- Processes start up fast.
- Processes shut down gracefully when requested.
- Processes are robust against sudden death
- Use MicroProfile or Spring annotation Fault Tolerance to make it resilient



From “CERN Data Centre Evolution”

X. Dev/prod parity

“Keep development, staging, and production as similar as possible”

- Development and production are as close as possible (in terms of code, people, and environments)
- Can use helm to deploy in repeatable manner
- Use (name)spaces for isolation of similar setups

XI. Logs

“Treat logs as event streams”

- App writes all logs to stdout
- Use a structured output for meaningful logs suitable for analysis. Execution environment handles routing and analysis infrastructure

XII. Admin processes

“Run admin/management tasks as one-off processes”

- Tooling: standard k8s tooling like “kubectl exec” or Kubernetes Jobs
- Also to be considered in solution/application design
- For example, if an application needs to migrate data into a database, place this task into a separate component instead of adding it to the main application code at startup

THE FACTORS

1. Codebase
2. Dependencies
3. Config
4. Backing Services
5. Build, Release, Run
6. Processes
7. Port binding
8. Concurrency
9. Disposability
10. Dev / Prod parity
11. Logs
12. Admin Processes


MicroProfile, Quarkus or Spring Config

Why?
– Configure Microservice without repacking the application How? – Specify the configuration in configure sources

– Access configuration via
- Programmatically lookup

```
Config config=ConfigProvider.getConfig();
config.getValue(“myProp”, String.class);
```

- Via CDI Injection

```
@Inject
@ConfigProperty(name="my.string.pro perty")
String myPropV;
```

Fault Tolerance

A solution to build a resilient microservice

- Retry - `@Retry`
- Circuit Breaker - `@CircuitBreaker`
- Bulk Head - `@Bulkhead`
- Time out - @`Timeout`
- Fallback - `@Fallback`

12 factor app

- Use MicroProfile, Spring or Quarkus and K8s to build a microservice => 12 factor app